

交易路径优化和执行编排的设计方案

MEV Solver: 算子规格说明书 (Operator Specifications)

编译流程:

Code block

```
1  编译流程:
2    输入: 原始交易图 (花瓣型)
3      ↓
4    【本文档范围】IR 优化算子
5      ├── Op_1: 拓扑优化 (边降级)
6      ├── Op_2: 虚拟化
7      ├── Op_3: 形态标记
8      ├── Op_4: 原语展开
9      └── Op_5: 结构修复
10     ↓
11    输出: ExecutionGraph (IR)
12      ├── 优化的拓扑 (Edge.FlowType)
13      ├── AST 结构 (Node.Form, Payload)
14      └── 执行顺序 (ExecOrder)
15     ↓
16    【下一阶段】编码
```

数据结构定义 (Core Data Structures)

1. 核心数据结构

Code block

```
1  // ExecutionGraph: 执行图 (中间表示 IR)
2  type ExecutionGraph struct {
3      Nodes      map[NodeID]*SwapNode
4      Edges      []*FundFlow
5      ExecOrder  []NodeID           // 执行顺序 (拓扑排序结果)
6      BotNode    NodeID             // Bot 节点 ID
7      PrincipalCap *big.Int        // 本金上限
8
9      // 运行时状态 (仅用于 Op_5)
```

```
10 RouterBalance map[TokenAddress]*big.Int // 模拟余额
11 }
```

Phase 1: 拓扑优化算子 (Topology Optimization Operators)

Op_1: Deterministic_Direct (确定性直连)

功能定义

消除 Router 中转，将满足条件的资金流从 $A \rightarrow \text{Router} \rightarrow B$ 优化为 $A \rightarrow B$ 。

接口签名

Code block

```
1 // 统一接口: Graph -> Graph 变换
2 func (op *Op1) Transform(graph *ExecutionGraph) (*ExecutionGraph, error)
3
4 // 返回: 修改后的新 Graph (原 Graph 不变, 遵循不可变原则)
5 // 元数据通过 graph.Context.Logger 记录
```

判定条件 (Decision Logic)

对于每条边 $E(A \rightarrow B)$ (通过 Router)，判定是否可直连：

Code block

```
1 Condition 1: 拓扑唯一性 (Topological Uniqueness)
2 Condition 2: 能力许可 (Capability Permission)
3 Condition 3: 资金覆盖 (Amount Coverage)
```

状态转换 (State Transformation)

Before (原始状态):

Code block

```
1 // Nodes
2 A: {ID: "uniV2_WETH_USDC", Out: 1000 USDC, CanRedirect: true}
3 B: {ID: "curve_USDC_DAI", In: 1000 USDC}
4
```

```
5 // Edges
6 E1: {From: A, To: Router, Token: USDC, FlowType: Transfer, GasCost: 23k}
7 E2: {From: Router, To: B, Token: USDC, FlowType: Transfer, GasCost: 23k}
8
9 // Total Gas: 46k
```

After (直连优化后):

Code block

```
1 // Nodes (不变)
2 A: {ID: "uniV2_WETH_USDC", Out: 1000 USDC, CanRedirect: true}
3 B: {ID: "curve_USDC_DAI", In: 1000 USDC}
4
5 // Edges (合并 + 类型降级)
6 E_new: {From: A, To: B, Token: USDC, FlowType: Direct, GasCost: 5k}
7
8 // Total Gas: 5k (节省 41k)
```

性能特征

- **时间复杂度:** $O(E^2)$ (E = 边数)
- **Gas 收益:** 每次应用节省 ~41k gas
- **成功率:** 实际场景中约 30%-50% 的边可直连

Op_2: Virtual_Reduction (虚拟规约)

功能定义

消除 **ERC20 Transfer**，将 Singleton 内部的资金流降级为虚拟边（仅记账，无实际转账）。

接口签名

Code block

```
1 // 统一接口: Graph -> Graph 变换
2 func (op *Op2) Transform(graph *ExecutionGraph) (*ExecutionGraph, error)
```

判定条件 (Decision Logic)

Code block

- 1 Condition 1: 必须已经是 Direct 边
- 2 Condition 2: 结算域一致 (Same Singleton)
- 3 Condition 3: 协议支持内部余额

状态转换 (State Transformation)

Before (直连状态):

Code block

```
1 // Nodes (都在 UniV4 Singleton 中)
2 A: {ID: "uniV4_pool_1", SingletonID: "0xUniV4Singleton", Out: 500 USDC}
3 B: {ID: "uniV4_pool_2", SingletonID: "0xUniV4Singleton", In: 500 USDC}
4
5 // Edges
6 E: {From: A, To: B, Token: USDC, FlowType: Direct, GasCost: 5k}
```

After (虚拟化后):

Code block

```
1 // Nodes (不变)
2 A: {ID: "uniV4_pool_1", SingletonID: "0xUniV4Singleton", Out: 500 USDC}
3 B: {ID: "uniV4_pool_2", SingletonID: "0xUniV4Singleton", In: 500 USDC}
4
5 // Edges (类型降级 + Gas 归零)
6 E: {From: A, To: B, Token: USDC, FlowType: Virtual, GasCost: 0}
```

性能特征

- 时间复杂度: $O(E)$
- Gas 收益: 每次应用节省 ~5k gas
- 前置依赖: 必须在 Op_1 之后执行

Phase 2: 结构实例化算子 (Structure Instantiation Operators)

Op_3: Mandatory_Scope (强制括号识别)

功能定义

根据协议特性进行 AST 转换，将 Callback 模式的节点标记为 `Scope`，保留 Standard 模式的节点为 `Node`。

接口签名

Code block

```
1 // 统一接口: Graph -> Graph 变换
2 func (op *Op3) Transform(graph *ExecutionGraph) (*ExecutionGraph, error)
3
4 // 返回: 修改后的新 Graph (所有节点的 Form 字段已设置)
```

判定条件 (Decision Logic)

Code block

```
1 func (op *Op3) DetermineForm(node *SwapNode) ExecutionForm {
2     switch node.Type {
3     case NodeTypeStandard: // V2, Curve
4         return FormNode
5     case NodeTypeCallback: // V3, V4, Balancer
6         return FormScope
7     case NodeTypeFlash: // Flash Loan
8         return FormScope
9     default:
10        return FormNode
11    }
12 }
```

状态转换 (State Transformation)

Before (平坦序列):

Code block

```
1 // ExecOrder
2 [A, B, C]
3
4 // Nodes (未设置 Form)
5 A: {Type: NodeTypeStandard, Form: nil} // UniV2
6 B: {Type: NodeTypeCallback, Form: nil} // UniV3
7 C: {Type: NodeTypeStandard, Form: nil} // Curve
```

After (形态标记):

```

1 // ExecOrder (不变)
Code block
2 [A, B, C]
3
4 // Nodes (设置 Form)
5 A: {Type: NodeTypeStandard, Form: FormNode, Payload: nil}
6 B: {Type: NodeTypeCallback, Form: FormScope, Payload: []} // 空 Payload
7 C: {Type: NodeTypeStandard, Form: FormNode, Payload: nil}

```

性能特征

- 时间复杂度: $O(N)$ (N = 节点数)
- Gas 影响: 无 (纯分类, 不改变执行逻辑)
- 前置依赖: 无

Op_4: Primitive_Conversion (原语展开)

功能定义

将 **Scope** 的回调语义展开为明确的执行顺序, 将抽象的 Scope 节点转换为三段式执行模式。

重要: 这是 **IR 层面的结构变换**, 不是最终的代码生成。输出仍然是 ExecutionGraph, 只是执行语义更明确。

接口签名

```

Code block
1 // 统一接口: Graph -> Graph 变换
2 func (op *Op4) Transform(graph *ExecutionGraph) (*ExecutionGraph, error)
3
4 // 返回: 修改后的新 Graph, ExecOrder 已展开为明确的三段式顺序

```

展开语义 (Expansion Semantics)

对于 Scope 节点, 其执行语义是三段式的:

```

Code block
1 Scope(B, Payload=[A, C]) 的执行语义:
2   1. Preamble: B 先乐观转账 (先给 Router 钱)
3   2. Body:      执行 Payload [A, C] (使用 B 的钱)
4   3. Postamble: B 回调时偿还 (Router 还钱给 B)

```

Op_4 将这个语义展开为 **扁平的执行顺序**:

Code block

```
1  // Before (嵌套结构)
2  ExecOrder: [NodeX, Scope(B, Payload=[A, C]), NodeY]
3
4  // After (扁平化, 语义明确)
5  ExecOrder: [
6      NodeX,
7      B_Preamble,    // 标记: B 的乐观转账阶段
8      A,              // Payload 执行
9      C,              // Payload 执行
10     B_Postamble,   // 标记: B 的偿还阶段
11     NodeY,
12 ]
```

状态转换 (State Transformation)

Before (Scope 嵌套结构):

Code block

```
1  // ExecOrder
2  [NodeA, ScopeB]
3
4  // ScopeB
5  B: {Form: FormScope, Payload: [C, D], TokenOut: USDC, AmountOut: 1000}
6  C: {Form: FormNode}
7  D: {Form: FormNode}
```

After (扁平化 + 阶段标记):

Code block

```
1  // 方案 1: 在 Node 上添加 Phase 标记
2  ExecOrder: [A, B, C, D, B]
3  Nodes: {
4      A: {Phase: Body}
5      B: {Phase: Preamble} // 第一次出现
6      C: {Phase: Body}
7      D: {Phase: Body}
8      B: {Phase: Postamble} // 第二次出现 (引用同一个 Node)
9  }
10
11 // 方案 2: 保持 Payload, 但添加执行阶段元数据
```

```

12 ExecOrder: [A, B] // 不变
13 B: {
14     Form: FormScope,
15     Payload: [C, D],
16     ExecutionPlan: {
17         Preamble: {Action: "OptimisticTransfer", Token: USDC, Amount: 1000},
18         Body: [C, D], // 按顺序执行
19         Postamble: {Action: "Repay", Token: USDC, Amount: 1000},
20     }
21 }

```

推荐实现: 保持结构，添加执行计划

Code block

```

1 // 在 SwapNode 中添加执行计划字段
2 type SwapNode struct {
3     // ... 原有字段 ...
4     Form          ExecutionForm
5     Payload       []NodeID
6
7     // Op_4 填充此字段 (仅当 Form=Scope 时)
8     ExecutionPlan *ScopeExecutionPlan
9 }
10
11 type ScopeExecutionPlan struct {
12     Preamble TransferAction // 乐观转账
13     Body      []NodeID      // 就是 Payload
14     Postamble TransferAction // 偿还
15 }
16
17 type TransferAction struct {
18     Type      string // "OptimisticTransfer" or "Repay"
19     Token     TokenAddress
20     Amount    *big.Int
21 }

```

性能特征

- **时间复杂度:** $O(N)$ (N = 节点数)
- **Gas 影响:** 每个 Scope 增加 ~2k gas (Preamble + Postamble)
- **前置依赖:** 必须在 Op_3 之后执行 (需要 Form 和 Payload)

Phase 3: 运行时修复算子 (Runtime Repair Operators)

Op_5: Engulfing (吞噬算子)

功能定义

偿付能力修复器，当模拟执行发现余额不足时，将缺钱的节点移入"富裕"的 Scope 节点的 Payload 中。

接口签名

Code block

```
1 // 统一接口: Graph -> Graph 变换
2 // MaxDepth 通过 graph.Context.MaxDepth 配置
3 func (op *Op5) Transform(graph *ExecutionGraph) (*ExecutionGraph, error)
4
5 // 返回: 修改后的新 Graph (ExecOrder 和 Payload 已调整)
6 // 如果修复失败, 返回 error
```

触发判定 (Trigger Logic)

在模拟执行过程中:

Code block

```
1 func (op *Op5) SimulateAndDetect(graph *ExecutionGraph) (needRepair bool, prey
  NodeID, token TokenAddress) {
2     balance := make(map[TokenAddress]*big.Int)
3
4     for _, nodeID := range graph.ExecOrder {
5         node := graph.Nodes[nodeID]
6
7         // 检查余额是否足够支付输入
8         if balance[node.TokenIn].Cmp(node.AmountIn) < 0 {
9             return true, nodeID, node.TokenIn // 发现偿付危机
10        }
11
12        // 更新余额
13        balance[node.TokenIn].Sub(balance[node.TokenIn], node.AmountIn)
14        balance[node.TokenOut].Add(balance[node.TokenOut], node.AmountOut)
15    }
16
17    return false, "", ""
18 }
```

吞噬策略 (Engulfing Strategies)

策略 A: 前向吞噬 (Forward Engulfing)

适用场景：Predator D 在 Prey N 之前执行。

状态转换 (Forward):

Code block

```
1 // Before
2 ExecOrder: [D(Scope), N(Node)]
3 D: {Payload: []}
4 N: {需要 100 USDC, 但 Router 余额为 0}
5
6 // After
7 ExecOrder: [D(Scope)] // N 已移除
8 D: {Payload: [N]} // N 被吞噬
```

策略 B: 后向吞噬 (Backward Engulfing)

适用场景：Prey N 在 Predator D 之前（逻辑上），需要提升 D 为根。

状态转换 (Backward):

Code block

```
1 // Before
2 ExecOrder: [N(Node), D(Scope)]
3 N: {需要 100 USDC, 但 Router 余额为 0}
4 D: {Payload: [], 产出 200 USDC}
5
6 // After
7 ExecOrder: [D(Scope)] // D 提升到前面, N 移除
8 D: {Payload: [N]} // N 被吞噬
```

递归吞噬示例 (Recursive Engulfing)

Code block

```
1 // 初始状态
2 ExecOrder: [A(Node), B(Node), C(Scope)]
3 RouterBalance: 0
4 A: {需要 50 USDC}
5 B: {需要 100 USDC}
```

```

6   C: {Form: Scope, 产出 200 USDC}
7
8   // 第一轮吞噬
9   // A 缺钱 → C 吞噬 A
10  ExecOrder: [B(Node), C(Scope)]
11  C: {Payload: [A]}
12
13  // 第二轮吞噬
14  // B 缺钱 → C 再次吞噬 B
15  ExecOrder: [C(Scope)]
16  C: {Payload: [A, B]} // 最终结构

```

性能特征

- **时间复杂度:** $O(N^2 \times A)$ (N = 节点数, A = 尝试次数)
- **Gas 影响:** 每次吞噬增加嵌套深度 ~2k gas
- **成功率:** 取决于是否存在合适的 Scope 节点
- **最坏情况:** 无法找到 Predator, 交易 Revert

IR 生成 Pipeline (IR Generation Pipeline)

完整的 IR 生成流程（输出中间表示，不生成最终指令）：

Code block

```

1  func GenerateIR(initialGraph *ExecutionGraph) (*ExecutionGraph, error) {
2      // 初始化上下文
3      initialGraph.Context = &OperatorContext{
4          PoolRegistry: NewPoolRegistry(),
5          GasModel:     NewGasModel(),
6          Logger:       NewLogger(),
7          MaxDepth:     3, // 最大嵌套深度
8      }
9
10     // IR 优化算子链
11     operators := []Operator{
12         &Op1_DeterministicDirect{}, // Phase 1: 直连优化
13         &Op2_VirtualReduction{},    // Phase 1: 虚拟化优化
14         &Op3_MandatoryScope{},      // Phase 2: 形态识别
15         &Op4_PrimitiveConversion{}, // Phase 2: 语义展开
16         &Op5_Engulfing{},           // Phase 3: 偿付修复
17     }
18
19     // 函数式组合: graph -> op1(graph) -> op2(graph) -> ...

```

```
20     graph := initialGraph
21     for _, op := range operators {
22         var err error
23         graph, err = op.Transform(graph)
24         if err != nil {
25             return nil, fmt.Errorf("operator %T failed: %w", op, err)
26         }
27     }
28
29     // 最终输出 (IR):
30     // - 优化后的拓扑结构 (graph.Edges with FlowType: Transfer/Direct/Virtual)
31     // - AST 形态 (graph.Nodes[*].Form, Payload)
32     // - 执行计划 (graph.Nodes[Scope].ExecutionPlan - 由 Op_4 填充)
33     // - 执行顺序 (graph.ExecOrder)
34     //
35     // 下一步: 短编码encoder消费这个 IR
36     return graph, nil
37 }
```

总结 (Summary)

算子特性对比

算子	类型	复杂度	Gas 影响	失败可能	依赖关系
Op_1	优化器	$O(E^2)$	-41k	无	无
Op_2	优化器	$O(E)$	-5k	无	Op_1 后执行
Op_3	解析器	$O(N)$	0	无	无
Op_4	展开器	$O(N)$	+2k/层	无	Op_3 后执行
Op_5	修复器	$O(N^2A)$	+2k/层	有	Op_3 后执行

关键工程考量

- Op_1/2 是纯优化：它们不依赖运行时状态，仅修改拓扑结构
- Op_3 是关键分界点：将协议特性转换为 AST 结构
- Op_5 是最后防线：仅在偿付危机时触发，有失败风险
- 深度限制至关重要：超过 3 层嵌套会导致 Gas 爆炸

5. 模拟器精度决定正确性：Op_5 依赖精确的余额模拟

IR 的价值 (Value of Intermediate Representation)

生成的 `ExecutionGraph` IR 包含：

1. 拓扑优化结果 (`Edges`):

- `FlowType: Transfer` - 需要通过 Router 中转
- `FlowType: Direct` - 直连，节省 ~41k gas
- `FlowType: Virtual` - Singleton 内部，0 gas

2. AST 结构 (`Nodes`):

- `Form: Node` - 原子执行
- `Form: Scope` - 回调作用域，包含 `Payload`
- `Payload` - 嵌套的子节点 ID 列表
- `ExecutionPlan` - (Op_4 填充) 三段式执行计划

3. 执行顺序 (`ExecOrder`):

- 拓扑排序后的节点序列
- Op_5 修复后的最终顺序

这套算子系统构成了一个**完整的 IR 优化器**，为后续短编码提供高质量的中间表示。